Modified Stoer-Wagner Algorithm for Image Segmentation via Volume-Normalized Global Minimum Cut

Aaron McKinstry
Oklahoma State University
IEM 5063: Network Optimization

May 8, 2026

Contents

1 Introduction

Graph-based image segmentation powers more everyday tools than most people realize. When you use Photoshop's "Select Subject" button, ask your phone to separate a person from their background, or use the magic wand tool to isolate an object, something in this family of techniques is often at work behind the scenes.

The core idea is to represent an image as a graph, where each pixel is a node and edges connect neighboring pixels with weights that reflect how similar those pixels are. Segmenting the image into meaningful regions such as the foreground or background then becomes a question of where to cut that graph. A good cut severs the fewest, weakest edges, corresponding to natural boundaries in the image.

Most classical approaches to this problem require the user to provide some guidance upfront, designating foreground and background seed pixels or sketching rough strokes to indicate what should be selected. This works well in interactive tools, but falls apart when you need fully automatic segmentation.

This project attempts to use the Stoer-Wagner global minimum cut algorithm [2] as an alternative. What makes it appealing is that it does not need a source, sink, or any input from the user. It finds the minimum cut of any undirected weighted graph in a single pass, making it a good candidate for this type of segmentation.

In practice, though, applying Stoer-Wagner directly to a pixel graph produces useless results, something I was not aware of at the time of the proposal. Given free rein, the algorithm tends to carve off a single isolated pixel rather than anything resembling a perceptual boundary. The bulk of this project is about understanding exactly why this happens and attempting to fix it.

The rest of this report works through these problems systematically. I start by characterizing why Stoer-Wagner fails on pixel graphs, tracing the issue to a mismatch between the raw cut objective and any reasonable notion of a perceptual boundary. From there, I implement a modified edge-weighting scheme and a volume-normalized scoring criterion that push the algorithm toward cuts that actually correspond to object boundaries. I do admit that my exact approach is not well documented as a standalone algorithm, but I treated this as an experience in adapting this to fit my use case.

2 Problem Statement

2.1 Image as a Graph

The starting point is converting an image into a graph. Given a grayscale image I of dimensions $R \times C$, with pixel intensities normalized to $[0, 1]$, we construct a weighted undirected graph $G = (V, E, w)$ where each pixel (r, c) becomes a node, edges connect each pixel to its 4-connected neighbors, and each edge is assigned a weight w reflecting how similar the two pixels are. The result is a grid graph with $|V| = RC$ nodes and $|E| \approx 2RC$ edges, slightly fewer at image boundaries where pixels have fewer neighbors.

2.2 The Global Minimum Cut Problem

A cut is a partition of V into two non-empty sets S and T, where $S \cup T = V$ and $S \cap T = \emptyset$. The cut weight $\text{cut}(S, T)$ is the total weight of edges crossing the partition, summing $w(u, v)$ over all edges (u, v) where $u \in S$ and $v \in T$.

The global minimum cut problem asks for the partition (S^*, T^*) that minimizes this quantity. The key distinction from the more familiar (s, t)-minimum cut is that no source or sink is specified. All possible ways of splitting the graph into two parts are implicitly considered, and the cheapest one wins.

2.3 What We Want

The goal is a partition (S, T) where the two sides correspond to something meaningful in the image, like a foreground object and the background. For that to work, the partition needs to satisfy three things. It should be reasonably balanced, so neither side is a single stray pixel. The cut should fall along actual intensity boundaries in the image rather than through homogeneous regions. And it should require no user input to compute.

3 Background

3.1 The Stoer-Wagner Algorithm

Stoer-Wagner [2] is a classical algorithm for finding the global minimum cut of an undirected weighted graph. It runs in $O(|V|^3)$ time, or $O(|V||E| + |V|^2 \log |V|)$ with a Fibonacci heap. The algorithm works in phases, running $|V| - 1$ of them in total. Each phase does two things: it finds a candidate cut using a procedure called maximum adjacency ordering, then it merges the last two vertices it found together into a single supernode. After all phases are done, the best candidate seen across all of them is the global minimum cut.

The maximum adjacency ordering (MAO) works by greedily building up a set A, starting from an arbitrary node. At each step it picks the node outside A that has the highest total edge weight connecting it to nodes already in A, and adds it. The last two nodes added, called s and t, define the cut-of-the-phase. The cut weight is the total weight of edges connecting t to everything else in the current graph. Stoer and Wagner proved that this is always a valid minimum (s, t)-cut, which is what makes the algorithm correct.

After recording the cut-of-the-phase as a candidate, the algorithm merges s and t into a single node, combining their edge weights to shared neighbors. This contraction is what allows the algorithm to implicitly consider all possible partitions without explicitly checking each one.

3.2 Related Work

The most closely related approach is the normalized cut of Shi and Malik [1]. Rather than minimizing raw cut weight, it minimizes:

$$\text{Ncut}(S,T) = \frac{\text{cut}(S,T)}{\text{vol}(S)} + \frac{\text{cut}(S,T)}{\text{vol}(T)}$$

where $\text{vol}(S)$ is the sum of all edge weights incident to nodes in S, counting edges to both sides of the partition. This normalization penalizes cuts that isolate small regions, since a tiny cluster has low volume and the ratio blows up. Minimizing Ncut exactly is NP-hard, so Shi and Malik reformulate it as an eigenvalue problem and solve the relaxed continuous version instead, then round the result back to a discrete partition.

4 The Degenerate Cut Problem

4.1 Failure of Naive Application

Applying Stoer-Wagner directly to a pixel adjacency graph with similarity-based edge weights almost always produces a useless result: a corner or edge pixel gets severed from the rest of the graph, yielding a partition of size 1 vs. $|V| - 1$. This is not a bug in the implementation. It is a fundamental consequence of the raw minimum cut objective applied to grid graphs where every node has at most four neighbors

4.2 Why This Happens

Take a 16×16 binary image with a white circle on a black background as a concrete example. With the standard similarity weight $w(u,v) = 1/(1 + |I_u - I_v|)$, a corner pixel sitting in the black background has two neighbors, both also black. The intensity difference between them is zero, so each edge gets weight $1/(1+0) = 1.0$. Cutting that corner pixel off from the rest of the graph costs a total of ≈ 2.0.

The cut we actually want separates the white circle from the black background. That boundary has roughly 50 edges, each crossing from a white pixel to a black pixel with an intensity difference of about 0.5, giving each edge a weight of $1/(1+0.5) \approx 0.67$. The total boundary cut weight is around $50 \times 0.67 = 33.5$, which is far more expensive than peeling off a corner. So the algorithm always prefers the corner, and the boundary cut never wins.

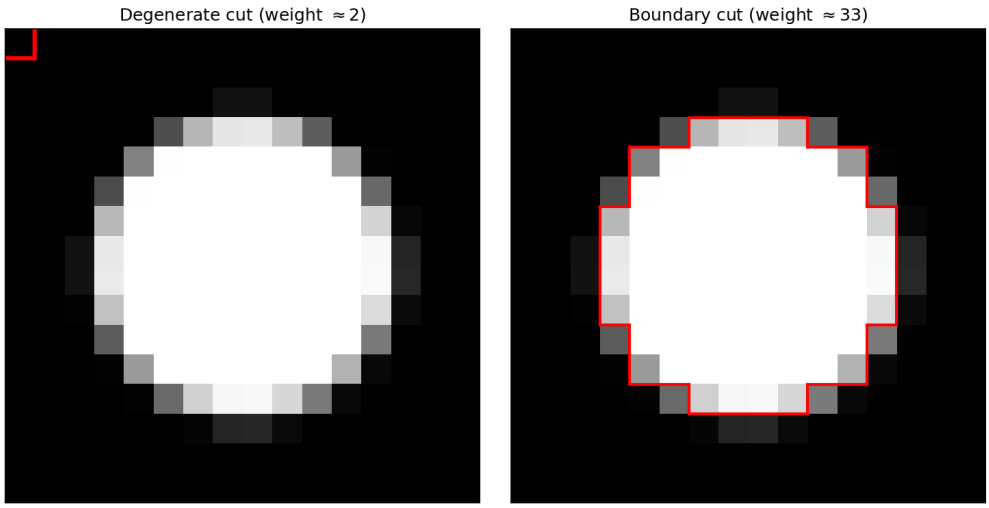

Figure 1: The corner pixel cut (weight ≈ 2) is always preferred by raw min-cut over the true boundary cut (weight ≈ 33).

5 Algorithmic Modifications

5.1 Modified Weight Function

The first change is to the edge weight function. The standard choice maps intensity difference to a similarity score in a narrow range, which is the root of the problem described in the previous section. Instead, we use:

$$w(u, v) = \frac{1}{\varepsilon + |I_u - I_v|}$$

with $\varepsilon = 0.01$. The effect is significant. Two pixels with identical intensity get an edge weight of $1/0.01 = 100$. Two pixels on opposite sides of a boundary, with an intensity difference of around 0.5, get a weight of about 2. Interior edges are now roughly 50 times more expensive to cut than boundary edges.

Going back to the circle example, the corner pixel cut now costs around 200 (two edges at weight 100 each), while the boundary cut costs around 100 (50 edges at weight 2 each). The boundary cut is cheaper, so the MAO will naturally start proposing it as a candidate.

5.2 Volume-Normalized Scoring

The second change is to how candidates are scored. Rather than keeping the cut with the lowest raw weight, we keep the one with the lowest volume-normalized score. The volume of a partition S is the total weight of all edges incident to nodes in S, counting edges on both sides:

$$\text{vol}(S) = \sum_{u \in S} \sum_{(u,v) \in E} w(u, v)$$

For each candidate cut (S, T) proposed by a phase, we compute:

$$\text{score}(S, T) = \frac{\text{cut}(S, T)}{\min(\text{vol}(S), \text{vol}(T))}$$

This is directly inspired by the Ncut objective of Shi and Malik [1]. A corner pixel has volume equal to its cut weight, so its score is always 1.0, the worst possible. A balanced boundary cut has a much larger volume on both sides relative to the cut weight, so it scores much lower.

5.3 Balance Constraint

As an additional safeguard, a candidate cut is only considered if:

$$\min(\text{vol}(S), \text{vol}(T)) \geq \alpha \cdot \text{vol}(V)$$

with $\alpha = 0.05$. This filters out cuts where one side accounts for less than 5% of the total graph volume. In practice the volume-normalized score already handles this, but the hard filter adds a useful safety net.

5.4 Complete Algorithm

For reference, the original Stoer-Wagner algorithm is shown first.

Algorithm 1 Stoer-Wagner (original)

```
while |V(g)| > 1 do
    run MAO on g to get cut-of-phase with weight c
    update best cut if c is smaller
    merge the last two nodes from MAO
end while
return best cut
```

The modified version makes three additions: it tracks which original nodes each supernode represents, scores candidates by volume-normalized cut weight rather than raw cut weight, and skips candidates that fail the balance constraint.

Algorithm 2 Modified Stoer-Wagner for Image Segmentation

```
while |V(g)| > 1 do
    run MAO on g to get cut-of-phase (s, t) with weight c
    compute score = c / min(vol(s side), vol(t side))
    if both sides exceed minimum volume and score is a new best then
        record current partition as best cut
    end if
    merge s and t, updating volumes
end while
return best cut
```

6 Implementation and Results

6.1 Structure

The implementation is split across three Python files. `read.py` takes a grayscale PNG, builds the pixel adjacency graph using NetworkX's `grid_2d_graph`, and assigns edge weights using the modified weight function. `stoer_wagner.py` contains the algorithm itself, broken into three functions: the outer loop, the MAO phase, and the merge step. `main.ipynb` ties everything together, handling image loading, downsampling, running the algorithm, and displaying the result as a binary mask. The full implementation is available at `https://github.com/Amckins/IEM5063-Network-Project`.

6.2 Downsampling

A 128×128 image produces 16,384 nodes, which is completely intractable for an $O(|V|^3)$ algorithm. All images are downsampled before building the graph. The default target is 16×16, which gives 256 nodes and runs in under a second. Lanczos resampling is used because it preserves edge sharpness better than simpler methods, which matters given that the weight function depends on intensity differences at boundaries.

6.3 Circle

The circle is the clearest success. The algorithm cleanly separates the white circle from the black background, with the partition boundary tracing the edge of the circle closely. A small number of anti-aliased boundary pixels end up on the wrong side, but visually the result is clean.

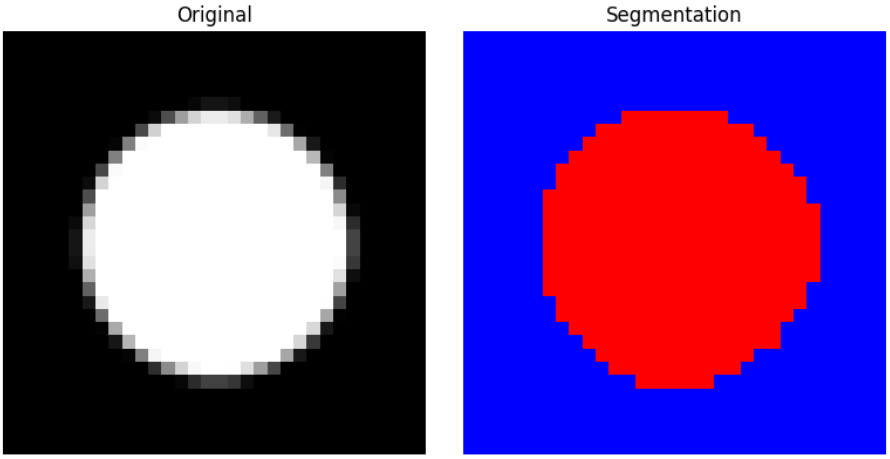

Figure 2: Circle: input (left) and segmentation result (right).

6.4 Star

The star works well despite its non-convex boundary. The algorithm traces most of the shape correctly. The only visible errors are near the tips of the points, which narrow to just a couple of pixels after downsampling and become harder to cut cleanly.

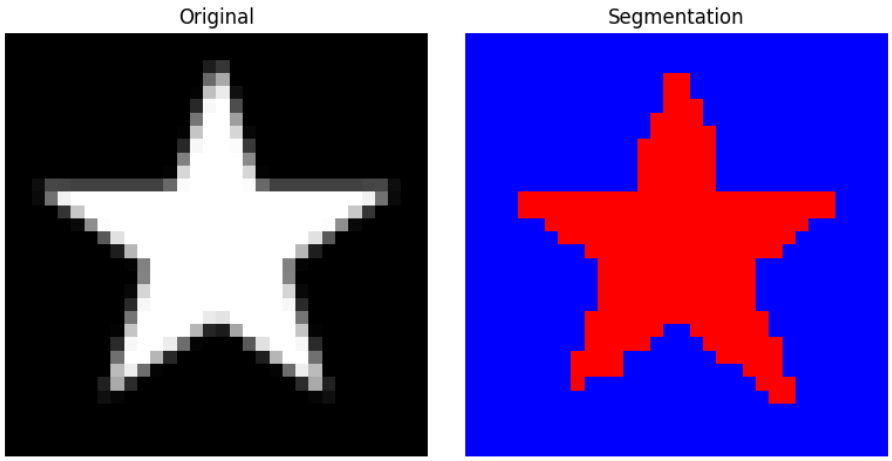

Figure 3: Star: input (left) and segmentation result (right).

6.5 Circle in Rectangle

The circle-in-rectangle exposes a structural limitation. The foreground has both an outer boundary and an inner boundary around the hole. A single cut cannot cleanly trace both at once, so the algorithm ends up cutting across one of them. The resulting partition does not correspond to anything meaningful in the image.

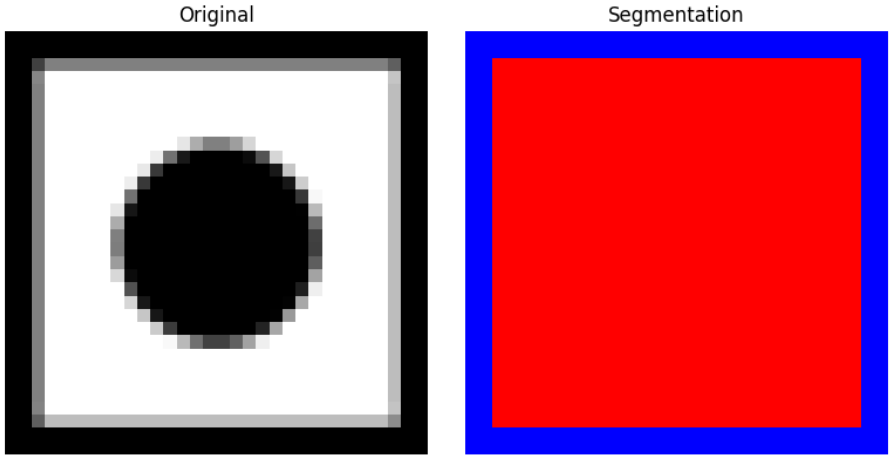

Figure 4: Circle in rectangle: input (left) and segmentation result (right).

6.6 Smiley Face

The smiley is the hardest case. The foreground consists of three disconnected components: two eyes and a mouth. The algorithm finds one cut and can isolate at most one of them at a time. In practice it tends to cut off a region of background rather than any of the foreground components, and the result bears no resemblance to the intended segmentation.

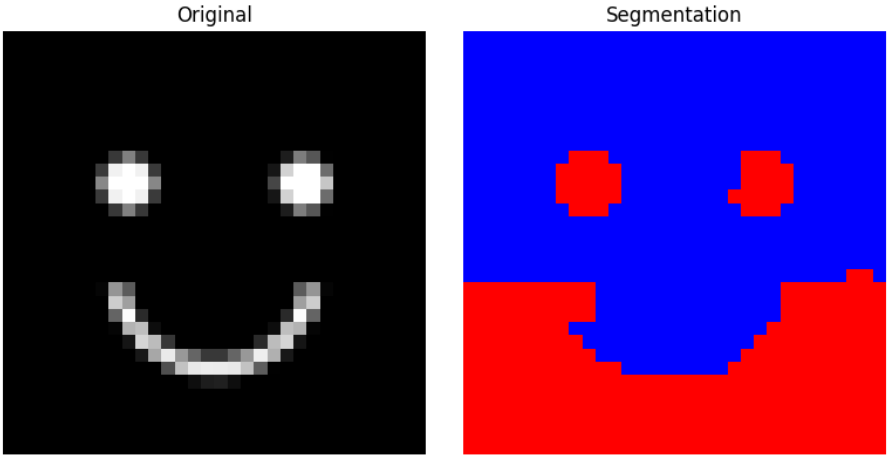

Figure 5: Smiley face: input (left) and segmentation result (right).

7 Discussion

7.1 Why the Modifications Work

The two changes work together. The modified weight function makes interior edges so much more expensive than boundary edges that the MAO starts proposing boundary cuts as natural candidates. Without that, the scoring change would not matter because the right cut would never come up as a candidate in the first place. Once the MAO is proposing boundary cuts, the volume-normalized score correctly identifies which of them is best. A single isolated pixel always scores 1.0 because its cut weight equals its volume, so it can never win over a balanced boundary cut that scores much lower.

The balance constraint adds a hard floor on top of this. In practice the score already handles degenerate cuts, so the constraint is somewhat redundant, but it provides a useful safety net and avoids computing scores on phases that are obviously too small to matter.

7.2 Limitations

The most significant limitation is that the algorithm finds exactly one cut. This is fine for images with a single connected foreground region, but fails immediately when the foreground has multiple disconnected parts. The smiley face result illustrates this directly: with three separate components, the best the algorithm can do is find a cut that happens to contain most of them on one side, which is not really a segmentation at all. Fixing this properly

would require running the algorithm recursively, splitting each partition and re-running until some stopping criterion is met.

The other hard limitation is scale. The $O(|V|^3)$ implementation becomes unusable above 32×32. A 64×64 image timed out entirely during testing. This is a Python implementation issue as much as an algorithmic one, but a Fibonacci heap version of the MAO would be needed to make larger images tractable.

Finally, the weight function is tuned specifically for binary images where intensity differences are either near zero or near one. Applying this to a natural grayscale image with gradual transitions would likely produce poor results without retuning ε or switching to a different weight function entirely.

8 Conclusion

Stoer-Wagner was not designed for image segmentation, and applying it directly does not work. The raw minimum cut objective will always find the cheapest way to split a graph, which on a pixel grid means peeling off a corner pixel rather than tracing any meaningful boundary. Getting it to do something useful required two changes: a weight function that makes interior edges far more expensive than boundary edges, and a volume-normalized scoring criterion borrowed from the normalized cut literature [1] that penalizes small isolated partitions.

With those changes in place, the algorithm works reasonably well on simple shapes with clear boundaries. The circle and star both segment cleanly on visual inspection. It breaks down predictably when the foreground is not simply connected, as with the circle-in-rectangle, and fails entirely when the foreground has multiple disconnected components, as with the smiley face. These are not surprising failures given that the algorithm produces a single cut, but they mark a hard boundary on what this approach can handle without further modification.

The most natural next step would be to apply the algorithm recursively, re-running it on each side of the partition until some stopping criterion is met. Whether the volume-normalized score provides a reliable stopping criterion is an open question. Scaling to larger images would also require a more efficient implementation, as the current $O(|V|^3)$ approach becomes intractable above 32×32.

References

[1] Jianbo Shi and J. Malik. Normalized cuts and image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(8):888–905, 2000.

[2] Mechthild Stoer and Frank Wagner. A simple min-cut algorithm. *Journal of the ACM*, 44(4):585–591, 1997.